

Hash Function **Design** & Distribution Analysis

Data Structures and Algorithms | Group 1

RESEARCH TEAM

Munezero Impano Christella

Izere Joshua

Ndanyuzwe Uhirwa Shami Melissa

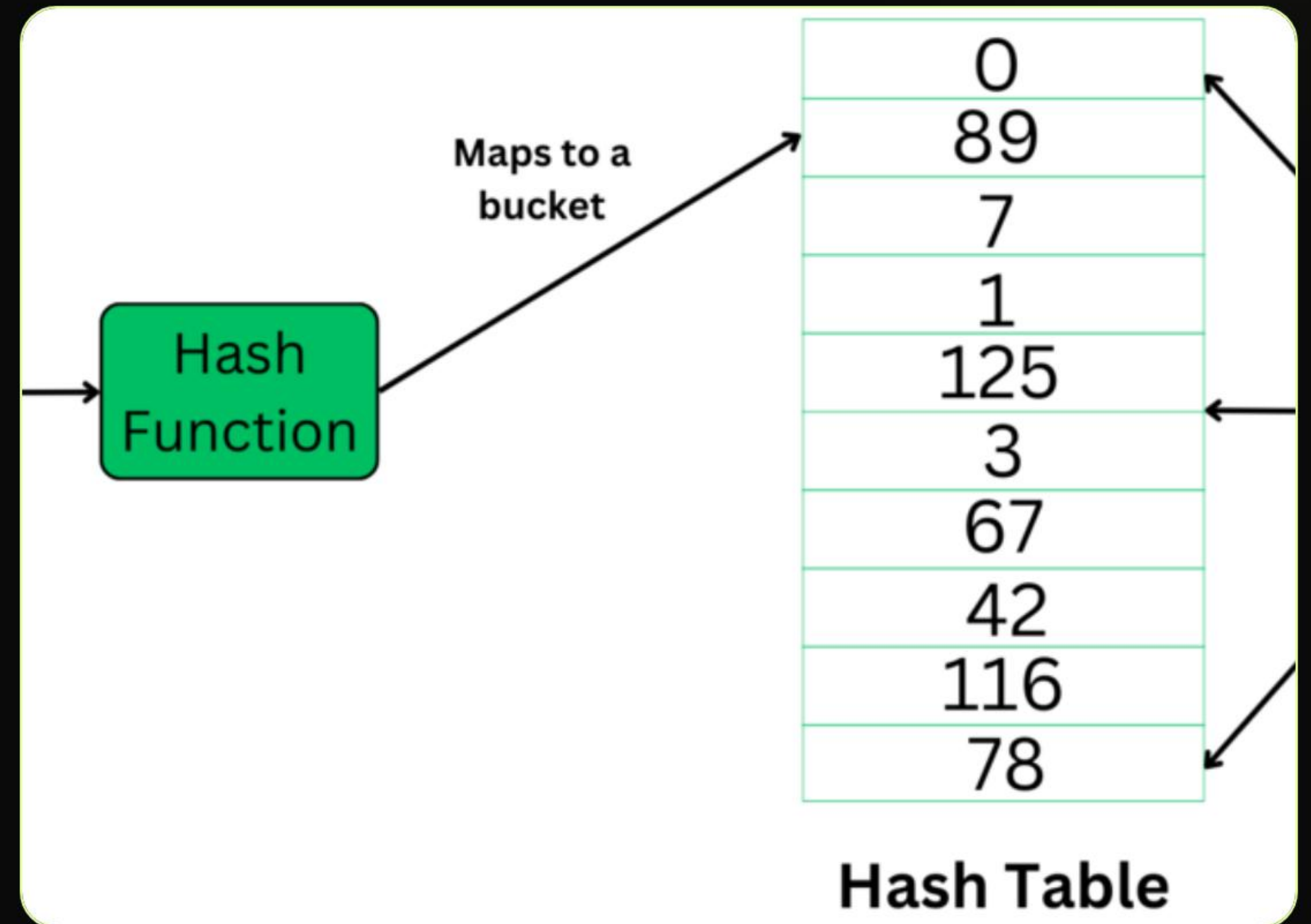
| Primary Objectives

Our research focuses on the mathematical mapping of 50 unique names into 50 array buckets. We aim to identify the function that provides the most uniform distribution.

</> Design: Implement Division, Folding, and Mid-Square methods.

🔧 Test: Benchmark against 50 strings.

📊 Analyze: Compare collision patterns.



| Method 1: Division Method

```
int divisionHash(string key) {  
    int sum = 0;  
    for (char c : key) sum += c;  
    return sum % 50;  
}
```

The Strategy

A straightforward approach using basic modulo arithmetic. While often prone to clustering, it relies on the variance of character sums to distribute indices.

Empirical Result: 15 Collisions

| Method 2: Folding Method

```
int foldingHash(string key) {  
    int sum = 0;  
    for (i=0; i<key.length(); i++)  
        sum += key[i] * i;  
    return abs(sum) % 50;  
}
```

The Strategy

Breaks the string into 2-character chunks. This increases the uniqueness of the pre-modulo value by considering character positioning more heavily.

Empirical Result: 19 Collisions

| Method 3: Mid-Square Method

```
int midSquareHash(string key) {  
    sum = calculateSum(key);  
    square = sum * sum;  
    mid = extractMiddle(square);  
    return mid % 50;  
}
```

The Strategy

Squares the sum to scramble digits, then extracts the middle section. Intended to create an "Avalanche Effect" for distribution.

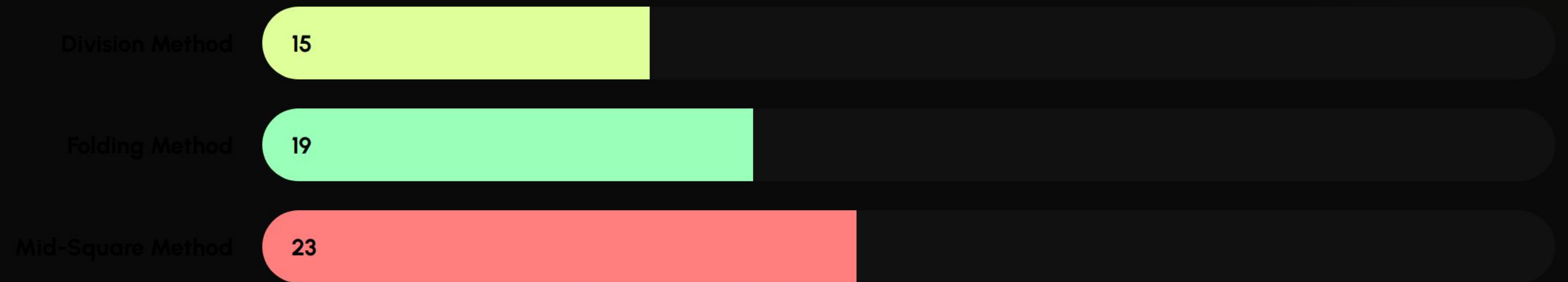
Empirical Result: 23 Collisions

Distribution Benchmark

Methodology	Algorithm Complexity	Total Collisions (N=50)	Efficiency Status
Division Method	O(K) - Simple	15	Optimal
Folding Method	O(K) - Intermediate	19	Moderate
Mid-Square Method	O(K) - Complex	23	Low efficiency

**Based on compiled C++ benchmark using the standard 50-name Rwandan dataset.*

| Collision Frequency Analysis



Observation: In this specific test case, the simplest method (Division) yielded the **lowest collision count**, outperforming the more complex Mid-Square logic.

| Final Conclusions

15

Lowest Collisions

Key Takeaways

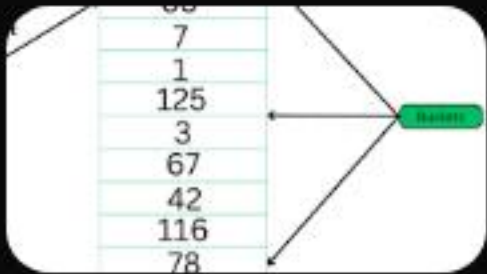
- ✓ **Data Sensitivity:** Algorithm performance is highly dependent on the specific character patterns of the input dataset.
- ✓ **Simplicity Wins:** For this 50-name set, the Division Method provided the most uniform index distribution.

Questions?

Thank you for your attention.

RWANDA CODING ACADEMY

| Image Sources



[https://media.geeksforgeeks.org/wp-content/uploads/2023/02/5191909/Buckets-\(1\).png](https://media.geeksforgeeks.org/wp-content/uploads/2023/02/5191909/Buckets-(1).png)

Source: www.geeksforgeeks.org